Practical No 2(AI)

2. Implement n-queens problem using Hill-climbing / simulated annealing / A* algorithm etc. Write a program for Water jug problem / Towers of Hanoi

```
# LP-II AI Practical

# 1. N-Queens (Hill Climbing)

# 2. Water Jug (BFS)

# 3. Tower of Hanoi (Recursion)


import random

……................................……..

def heuristic(state):

    attacks = 0

    n = len(state)

    for i in range(n):

        for j in range(i + 1, n):

            if state[i] == state[j] or abs(state[i] - state[j]) == j - i:

                attacks += 1

    return attacks

……..................................................……..

def random_state(n):

    return [random.randint(0, n - 1) for _ in range(n)]

……..............................................................……..
```

```
def hill_climbing(n):
    current = random_state(n)
    current_h = heuristic(current)

    while True:
        neighbors = []
        for col in range(n):
            for row in range(n):
                if row != current[col]:
                    neighbor = current.copy()
                    neighbor[col] = row
                    neighbors.append(neighbor)

        next_state = current
        next_h = current_h

        for neighbor in neighbors:
            h = heuristic(neighbor)
            if h < next_h:
                next_state = neighbor
                next_h = h
```

```
        if next_h >= current_h:
            break

        current, current_h = next_state, next_h

    return current, current_h
```

……..

```
solution, conflicts = hill_climbing(8)
print("Solution:", solution)
print("Number of Conflicts:", conflicts)
```

2. Water Jug (BFS)

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])
    path = []
```

```
while queue:
    x, y = queue.popleft()

    if (x, y) in visited:
        continue

    visited.add((x, y))
    path.append((x, y))

    if x == target or y == target:
        print("Solution Path:")
        for step in path:
            print(step)
        return

    queue.extend([
        (jug1, y),       # Fill jug1
        (x, jug2),       # Fill jug2
        (0, y),          # Empty jug1
        (x, 0),          # Empty jug2
        (min(jug1, x+y), max(0, x+y-jug1)),  # Pour jug2 → jug1
```

```
            (max(0, x+y-jug2), min(jug2, x+y))   # Pour jug1 → jug2
        ])

    print("No solution")
```

……

```
water_jug(4, 3, 2)
```

……..

```
# 3. Tower of Hanoi (Recursion)
def towers_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    towers_of_hanoi(n - 1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    towers_of_hanoi(n - 1, auxiliary, source, destination)
```

……

```
towers_of_hanoi(3, 'A', 'B', 'C')
```

Untitled Last Checkpoint: 50 minutes ago

```
        queue.extend([
            (jug1, y),       # Fill jug1
            (x, jug2),       # Fill jug2
            (0, y),          # Empty jug1
            (x, 0),          # Empty jug2
            (min(jug1, x+y), max(0, x+y-jug1)),  # Pour jug2 → jug1
            (max(0, x+y-jug2), min(jug2, x+y))   # Pour jug1 → jug2
        ])

    print("No solution")
```

```
Solution: [5, 2, 6, 1, 7, 4, 0, 3]
Number of Conflicts: 0
```

[19]:
```
water_jug(4, 3, 2)
```

```
Solution Path:
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
```

[16]:
```
# 3. Tower of Hanoi (Recursion)
def towers_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
```

Untitled Last Checkpoint: 52 minutes ago

```
(1, 0)
(3, 3)
(0, 1)
(4, 2)
```

[16]:
```
# 3. Tower of Hanoi (Recursion)
def towers_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    towers_of_hanoi(n - 1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    towers_of_hanoi(n - 1, auxiliary, source, destination)
```

[17]:
```
towers_of_hanoi(3, 'A', 'B', 'C')
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

[ ]: